AREHI-SECOPS

ASEAN Regional E-Commerce Hybrid Infrastructure & Hardened SecOps System

Phase 4 — AWS Cloud Infrastructure

Full-Depth Terraform & Certification Study Guide

Architect / Author: Danial Nur Irfan bin Azeze
Bachelor of Computer Science — Networks & Security, UTM
CompTIA Security+ Certified

Exam objectives referenced: AWS Certified Solutions Architect – Associate (SAA-C03), CompTIA Security+ SY0-701, Cisco CCNA 200-301
Tooling: Terraform (HashiCorp AWS provider ~> 5.0), written and reviewed — never applied

Region: ap-southeast-1 (Singapore)
Resources in scope: VPC, subnets, NAT Gateways, Security Groups/NACLs, ALB, Auto Scaling Group, RDS Multi-AZ + read replica, Secrets Manager, AWS WAF, KMS + S3 audit logging, Site-to-Site VPN

Contents

1. Phase 4 Overview

Phase 4 moves the project from on-premises networking (Phases 1–3) into the public cloud, building the AWS side of the hybrid infrastructure the earlier phases were always designed to connect to — recall `SG-EDGE-GW`'s placeholder WAN interface (`203.0.113.0/24`) and its NAT/ACL configuration in Phase 3, both written in anticipation of exactly this. Where Phases 1–3 were typed by hand into a live Packet Tracer lab and verified with `show` commands, Phase 4 is **Terraform** — Infrastructure as Code — written to be complete and deployable, and reviewed line by line for correctness, but deliberately never run against a real AWS account (see Section 3).

Resource files in scope

File	AWS resources	Primary purpose
`vpc.tf`	VPC, 6 subnets, IGW, 2 NAT Gateways, route tables	Network foundation — 3-tier subnet design across 2 AZs
`security-groups.tf`	3 Security Groups, 3 NACLs	Defense in depth — stateful + stateless traffic control per tier
`alb.tf`	ALB, target group, listeners, IAM role, launch template, ASG	Internet-facing, auto-scaling web/app tier
`rds.tf`	RDS primary (Multi-AZ), read replica, Secrets Manager	Highly-available relational database tier
`waf.tf`	WAFv2 Web ACL + association	Layer 7 application-layer attack filtering
`logging.tf`	KMS key, S3 bucket, bucket policy	Encrypted, immutable audit log storage
`vpn-gateway.tf`	Customer Gateway, VPN Gateway, VPN Connection	Hybrid connectivity back to the ASEAN on-prem network
`variables.tf` / `outputs.tf`	Input variables, output values	Shared configuration surface across every file above

2. A Note on Exam Objective Tags

Phase 4 flips the tag distribution seen in Phases 1–3: there, CCNA and Security+ dominated and AWS SAA-C03 only appeared where the on-prem design touched a cloud concept. Here, **AWS SAA-C03** is the primary certification almost every objective maps to directly — this phase covers SAA-C03 Domain 1.0 (Design Resilient Architectures: Multi-AZ, Auto Scaling, read replicas), Domain 2.0 (Design High-Performing Architectures: ALB, NAT Gateway placement), Domain 3.0 (Design Secure Applications and Architectures:

Security Groups/NACLs, WAF, KMS, Secrets Manager, IAM roles), and Domain 4.0 (Design Cost-Optimized Architectures, referenced in the sizing decisions in `variables.tf`). **Security+ SY0-701** remains present throughout — this phase is effectively a continuous demonstration of Domain 3.0 (Security Architecture) applied to a real cloud topology. **CCNA 200-301** appears only once, directly, in the Site-to-Site VPN objective, which is a literal continuation of the WAN/hybrid-connectivity concepts from Phase 1.

3. A Note on Scope — Why This Is Never Applied

Decision: this Terraform is written to be complete, internally consistent, and of deployable quality — but no `terraform apply` is ever run against a real AWS account as part of this project.

> **WHY, STATED EXPLICITLY**
> There is no live AWS account behind this work, and actually deploying this configuration would incur real, ongoing hourly costs regardless of traffic volume — a NAT Gateway per AZ, an RDS Multi-AZ instance plus a read replica, ALB capacity units, and running EC2 instances all bill continuously the moment they exist, not just when used. This mirrors exactly how the on-premises phases treated `SG-EDGE-GW`'s AWS-facing placeholder interface (`203.0.113.0/24`, an RFC 5737 documentation address) — represented accurately and taken seriously as a design exercise, never assumed to be a live, reachable endpoint.

A direct consequence: the Site-to-Site VPN in Objective 8 configures the **AWS side** of the tunnel completely and correctly (Customer Gateway, VPN Gateway, VPN Connection), but it can never actually negotiate a live tunnel in this project, because its peer — `SG-EDGE-GW`'s placeholder public IP — was never going to be reachable on the real internet in the first place. The gap is the lab's inherent boundary, not a mistake in either phase's configuration.

3.1 LocalStack Local Validation Results

Never applying to real AWS doesn't mean never testing at all. This Terraform was additionally run against **LocalStack** — a free, local AWS emulator (Docker-based) — specifically to catch real configuration bugs that `terraform plan` alone can't surface, without incurring any real AWS cost. Throughout the objective sections below, code blocks affected by this testing are marked with ~~strikethrough~~ and an italic note explaining why.

Result	What
Verified with a real `terraform apply`	VPC, all 6 subnets, Security Groups + rules, NACLs, KMS key, S3 audit bucket (encryption/versioning/lifecycle/policy), IAM role + launch template, Secrets Manager secret, and the full Customer Gateway → VPN Gateway → VPN Connection chain — confirmed live via `aws ec2 describe-*` / `aws s3api` calls against the LocalStack endpoint, not just a successful exit code.
Blocked - LocalStack Community license	`elbv2` (ALB, target group, both listeners, and by extension the ASG/scaling policy which depend on the target group's ARN) and `rds` (DB subnet group, both DB instances) and `wafv2` (Web ACL + association) all returned "service is not included within your LocalStack license" - a LocalStack Pro-tier restriction, not a flaw in the Terraform.
Blocked - not implemented in LocalStack	`aws_vpn_connection_route` and both `aws_vpn_gateway_route_propagation` resources returned "action has not been implemented," independent of license tier.

A REAL BUG THIS TESTING ACTUALLY CAUGHT

The original `security-groups.tf` mixed inline `ingress` / `egress` blocks on `aws_security_group` with separate `aws_security_group_rule` resources targeting the same security group - a genuine Terraform/AWS-provider anti-pattern. A live `terraform plan` against LocalStack showed `aws_security_group.app` about to silently delete its own 5432-to- `db-sg` egress rule on every apply, which would have broken database connectivity had this ever been applied to real AWS. This was not a LocalStack artifact - it would have happened identically on real AWS - and was fixed permanently by moving every rule into its own `aws_security_group_rule` resource. See `03-aws-cloud-infrastructure/security-groups/README.md` for the full explanation.

4. Objective 1 — Planning: VPC Addressing & Resource Inventory

Planning only — `phase4-plan.md`, produced before any Terraform was written

AWS SAA-C03: Domain 1.0 (Design Resilient Architectures)

Purpose: the same "design first, then build" discipline used for the IPv4/IPv6 addressing plans in Phases 1–2 — decide the VPC's CIDR breakdown and every file's intended contents before writing a single `resource` block.

VPC addressing plan

Tier	AZ ap-southeast-1a	AZ ap-southeast-1b	Purpose
Public	10.200.0.0/24	10.200.1.0/24	ALB, NAT Gateways — internet-facing
Private App	10.200.10.0/24	10.200.11.0/24	EC2 web/app tier (Auto Scaling Group)
Private DB	10.200.20.0/24	10.200.21.0/24	RDS Multi-AZ primary + read replica

Parent CIDR `10.200.0.0/16`, region `ap-southeast-1` (Singapore — geographically consistent with `SG-EDGE-GW` being the on-prem side of the hybrid connection). Each tier gets a full `/24`, and the third octet's tens digit doubles as a tier identifier (`0` / `1` = public, `10` / `11` = app, `20` / `21` = DB) — the same VLSM-by-convention logic the on-prem VLAN plan used.

> **EXAM ANGLE — WHY 2 AZS, NOT 1**
>
> An Availability Zone is one or more physically distinct data centers with independent power, cooling, and networking within a region. Spanning exactly 2 AZs is the minimum required to satisfy "Multi-AZ" in any real SAA-C03 scenario — a design confined to 1 AZ has no resilience against an entire-data-center failure, no matter how many resources exist inside it.

5. Objective 2 — VPC, Subnets, Internet Gateway, NAT Gateways, Route Tables

`vpc.tf`

AWS SAA-C03: Domain 1.0 & 2.0 (Resilient & High-Performing Architectures)

Purpose: the network foundation every other resource in this phase attaches to — a VPC subdivided into 3 tiers across 2 AZs, with internet reachability handled differently per tier depending on what that tier actually needs.

Resource blocks, word by word

```
resource "aws_vpc" "main" {
  cidr_block           = var.vpc_cidr
  enable_dns_support   = true
  enable_dns_hostnames = true
}

resource "aws_internet_gateway" "main" {
  vpc_id = aws_vpc.main.id
}
```

Argument	Meaning
`enable_dns_support`	Lets instances inside the VPC resolve DNS at all, using the VPC's internal DNS resolver — required for essentially everything else (RDS endpoints, Secrets Manager, the ALB's own DNS name) to be reachable by name rather than raw IP.
`enable_dns_hostnames`	Additionally assigns public DNS hostnames to instances that have public IPs — needed for the public subnet tier specifically.
Internet Gateway (IGW)	A horizontally-scaled, redundant VPC component (not a single device to size or fail over) that provides the actual path between the VPC and the internet. Exactly one per VPC; every public subnet's route table points default traffic at it.

Public subnets, NAT Gateways — one per AZ, word by word

```
resource "aws_subnet" "public" {
  count                   = length(var.public_subnet_cidrs)
  availability_zone       = var.availability_zones[count.index]
  map_public_ip_on_launch = true
}

resource "aws_eip" "nat" {
  count  = length(var.availability_zones)
  domain = "vpc"
}

resource "aws_nat_gateway" "main" {
  count         = length(var.availability_zones)
  allocation_id = aws_eip.nat[count.index].id
  subnet_id     = aws_subnet.public[count.index].id
}
```

Term	Meaning
`count = length(...)`	Terraform's loop construct — creates one resource per list element, here producing exactly 2 of each (one per AZ) without repeating the whole block by hand. This is the idiomatic Terraform pattern for "one of these per AZ."
`map_public_ip_on_launch`	Any instance launched into this subnet automatically receives a public IP — appropriate only for the public tier; the app and DB subnets deliberately omit this entirely.
NAT Gateway	Lets instances in a *private* subnet (no public IP of their own) initiate outbound internet connections — for OS patching, package installs — while remaining completely unreachable from the internet inbound. This is the managed, AWS-operated equivalent of the PAT/overload NAT configured by hand on `SG-EDGE-GW` in Phase 3.
Elastic IP (EIP)	A static public IPv4 address, allocated (`domain = "vpc"`) specifically so each NAT Gateway has a fixed, unchanging address for the app tier's outbound traffic to originate from.

EXAM ANGLE — ONE NAT GATEWAY PER AZ, NOT ONE SHARED

A single NAT Gateway is a real single point of failure: if its AZ has a problem, *every* private subnet across the whole VPC loses outbound internet access simultaneously, regardless of how many AZs the rest of the design spans. SAA-C03 tests this exact tradeoff directly — "one NAT Gateway per AZ" is the correct default answer whenever a scenario asks about NAT Gateway resilience, and it's exactly why `vpc.tf` loops over `var.availability_zones` rather than creating just one.

App & DB route tables — per-AZ routing, word by word

```
resource "aws_route_table" "app" {
  count = length(var.availability_zones)

  route {
    cidr_block     = "0.0.0.0/0"
    nat_gateway_id = aws_nat_gateway.main[count.index].id
  }
  route {
    cidr_block = var.on_prem_cidr
    gateway_id = aws_vpn_gateway.main.id
  }
}

resource "aws_route_table" "db" {
  route {
    cidr_block = var.on_prem_cidr
    gateway_id = aws_vpn_gateway.main.id
  }
}
```

Detail	Meaning
Per-AZ app route tables (`count = length(var.availability_zones)`)	Each AZ's app subnet routes its default (internet-bound) traffic through *that same AZ's own* NAT Gateway — never across an AZ boundary. This keeps the two AZs independently resilient: if AZ-b's NAT Gateway or entire AZ has a problem, AZ-a's app subnet is completely unaffected.
DB route table — no `0.0.0.0/0` route at all	The single most consequential line in this file: there is no default route, no NAT Gateway route, no IGW route for the DB tier — full stop. A database has no legitimate reason to ever initiate an outbound internet connection, and omitting the route entirely removes a whole class of exfiltration risk rather than relying on a Security Group rule to block it after the fact.
VPN Gateway route (both app and DB tables)	The counterpart to the Site-to-Site VPN in Objective 8 — lets both tiers reach the on-prem `10.10.0.0/16` network (all 4 ASEAN sites) over the encrypted tunnel, not just the internet.

EXPECTED RESULT (WERE THIS EVER APPLIED)

```
terraform validate
terraform plan
```

> `terraform validate` confirms the configuration is syntactically valid and internally consistent (every referenced resource actually exists); `terraform plan` would show 1 VPC, 1 IGW, 6 subnets, 2 EIPs, 2 NAT Gateways, and 4 route tables (1 public, 2 app, 1 DB) queued for creation, with zero resources to destroy or modify on a first run.

6. Objective 3 — Security Groups & Network ACLs

`security-groups.tf` — full rule matrix in `03-aws-cloud-infrastructure/security-groups/README.md`

AWS SAA-C03: Domain 3.0 (Design Secure Applications and Architectures) | Security+ SY0-701: Domain 3.0 (Security Architecture)

Purpose: two independent layers of traffic control, deliberately both used rather than just one, specifically to demonstrate the stateful-vs-stateless distinction both AWS SAA-C03 and Security+ test directly. Trust model: each tier trusts only the one tier immediately in front of it — nothing trusts the raw internet except the ALB's 443 listener, and the DB tier trusts nothing but the app tier's own Security Group.

Security Groups — stateful, word by word

```
resource "aws_security_group" "app" {
  ingress {
    from_port       = 8443
    to_port         = 8443
    protocol        = "tcp"
    security_groups = [aws_security_group.alb.id]
  }
}

resource "aws_security_group_rule" "app_to_db_egress" {
  type                     = "egress"
  from_port                = 5432
  to_port                  = 5432
  security_group_id        = aws_security_group.app.id
  source_security_group_id = aws_security_group.db.id
}
```

Term	Meaning
Stateful	A Security Group rule only ever needs to be written in **one** direction. Traffic permitted inbound automatically has its response traffic permitted back outbound, regardless of what the egress rules say — this is the core exam-testable property distinguishing a Security Group from a NACL.
`security_groups = [aws_security_group.alb.id]`	Referencing another Security Group's *ID* as the source, instead of a CIDR block. This is what lets the rule stay correct automatically as the Auto Scaling Group launches and terminates instances with different private IPs — group membership is evaluated, not a fixed address range.
`aws_security_group_rule` as a separate resource	Split out from the inline block specifically to reference `aws_security_group.db`, which is declared later in the same file — an inline rule inside `app`'s own block referencing `db` (which itself would need to reference `app`) would create a circular dependency Terraform cannot resolve.
DB Security Group has no egress rule at all	Same reasoning as the DB route table in Objective 2 — a database has no legitimate reason to initiate outbound connections, so the Security Group's implicit "deny all egress" is left in place rather than adding a permissive rule defensively.

Network ACLs — stateless, word by word

```
resource "aws_network_acl" "db" {
  ingress {
    rule_no    = 100
    protocol   = "tcp"
    action     = "allow"
    cidr_block = var.vpc_cidr
    from_port  = 5432
    to_port    = 5432
  }
  ingress {
    rule_no    = 110
    cidr_block = var.vpc_cidr
    from_port  = 1024
    to_port    = 65535
  }
}
```

Term	Meaning
Stateless	Return traffic is **not** automatically permitted — every NACL in this file needs an explicit ephemeral-port rule (1024–65535) in both directions, or a connection's response traffic gets silently dropped even though the request itself was allowed through.
`rule_no`	NACL rules are evaluated in ascending numeric order, and the **first** match wins — unlike a Security Group, where every matching rule is simply permissive and order doesn't matter. Numbering with gaps (100, 110...) leaves room to insert a rule later without renumbering everything.
Ephemeral port range (1024–65535)	When a client opens a connection, its *source* port is a randomly chosen high-numbered port, not the well-known destination port (5432, 443, etc). The NACL must permit traffic *to* that ephemeral range for return traffic to actually arrive back at the client — this has no equivalent concept in a stateful Security Group.
`cidr_block = var.vpc_cidr`, not a security group reference	NACLs cannot reference a Security Group ID at all — only CIDR blocks. This is the concrete, testable reason NACLs alone could never fully replace Security Groups in this design: the DB NACL can only ever say "allow from the whole VPC," not "allow from the app tier specifically," which is exactly what the DB Security Group achieves instead.

EXAM ANGLE — DEFENSE IN DEPTH, NOT REDUNDANCY

If the DB Security Group were ever misconfigured too permissively, the DB subnet's NACL — which only permits VPC-CIDR traffic on 5432 — would still block any connection attempt originating from outside the VPC entirely. Neither layer is "the" control; the pairing of both is the control, and SAA-C03 scenario questions frequently hinge on recognizing that a NACL fix and a Security Group fix solve different failure modes.

EXPECTED RESULT (WERE THIS EVER APPLIED)

Each tier's Security Group would show exactly one narrowly-scoped ingress rule sourced from the tier in front of it (never a CIDR, except at the internet-facing ALB); each NACL would show explicit numbered allow rules for both the service port and the ephemeral range, in both directions.

7. Objective 4 — Application Load Balancer & Auto Scaling Group

`alb.tf`

AWS SAA-C03: Domain 1.0 & 2.0 (Resilient & High-Performing Architectures)

Purpose: the internet-facing entry point for the whole application, terminating HTTPS, distributing traffic across an Auto Scaling Group of EC2 instances, and scaling that group automatically under load — without a fixed number of servers ever being hand-provisioned.

ALB & listeners, word by word

```
resource "aws_lb" "main" {
  load_balancer_type = "application"
  security_groups    = [aws_security_group.alb.id]
  subnets            = aws_subnet.public[*].id

  access_logs {
    bucket  = aws_s3_bucket.audit_logs.id
    enabled = true
  }
}

resource "aws_lb_listener" "https" {
  port            = 443
  protocol        = "HTTPS"
  ssl_policy      = "ELBSecurityPolicy-TLS13-1-2-2021-06"
  certificate_arn = var.acm_certificate_arn
}

resource "aws_lb_listener" "http_redirect" {
  port = 80
  default_action {
    type = "redirect"
    redirect { port = "443", protocol = "HTTPS", status_code = "HTTP_301" }
  }
}
^ elbv2 gated behind LocalStack's paid tier - verified only via `terraform plan`, not `apply`.
See Section 3.1.
```

Term / argument	Meaning
Application Load Balancer (Layer 7)	Operates at the HTTP/HTTPS layer specifically — unlike a Network Load Balancer (Layer 4), it can inspect and route on request content, which is also exactly what lets AWS WAF (Objective 6) attach to it and inspect requests before they ever reach an instance.
`subnets = aws_subnet.public[*].id`	The ALB itself lives in the public subnets (one ENI per AZ, automatically), even though the actual application instances behind it live in the private app subnets — the ALB is the only thing in this design that legitimately needs a foot in both worlds.
`ssl_policy = "ELBSecurityPolicy-TLS13-1-2-2021-06"`	An AWS-managed predefined set of allowed TLS protocol versions and cipher suites — this specific policy requires TLS 1.2 at minimum while preferring 1.3, rejecting older, weaker TLS versions automatically rather than the operator having to enumerate ciphers by hand.
`certificate_arn = var.acm_certificate_arn`	References an AWS Certificate Manager-issued certificate. Declared as a variable with an obviously-fake placeholder ARN rather than hardcoded, since no real ACM certificate has ever been issued for this project.
HTTP→HTTPS redirect listener	Port 80 exists on the ALB purely to catch stray plaintext requests and immediately 301-redirect them to 443 — no application traffic is ever actually served over plaintext HTTP.
ALB access logs → S3	Every request the ALB handles is logged to the KMS-encrypted bucket from Objective 7, giving a durable audit trail independent of anything happening at the application layer.

IAM role, launch template, ASG, word by word

```
resource "aws_iam_role_policy" "app_read_db_secret" {
  policy = jsonencode({
    Statement = [{
      Effect   = "Allow"
      Action   = ["secretsmanager:GetSecretValue"]
      Resource = [aws_secretsmanager_secret.db_credentials.arn]
    }]
  })
}

resource "aws_launch_template" "app" {
  iam_instance_profile { arn = aws_iam_instance_profile.app.arn }
}

resource "aws_autoscaling_group" "app" {
  vpc_zone_identifier = aws_subnet.app[*].id
  min_size            = var.asg_min_size
  max_size            = var.asg_max_size
  health_check_type   = "ELB"
}

resource "aws_autoscaling_policy" "scale_out_on_cpu" {
  policy_type = "TargetTrackingScaling"
  target_tracking_configuration {
    predefined_metric_specification { predefined_metric_type = "ASGAverageCPUUtilization" }
    target_value = 60.0
  }
}
^ blocked by the target group above being unavailable (elbv2 gated) - the ASG/policy APIs
themselves are free-tier, but target_group_arns has nothing to reference. IAM role/policy and
the launch template above verified with a real `apply`.
```

Term	Meaning
IAM role scoped to exactly one action, one resource	The EC2 instance role can only call `GetSecretValue`, and only on the one specific secret ARN it actually needs — not `secretsmanager:*`, not a wildcard resource. This is the **principle of least privilege** applied concretely, the same concept tested throughout Security+ Domain 3.0/5.0 and SAA-C03's security domain alike.
Instance profile	The mechanism that actually attaches an IAM role to an EC2 instance — a role by itself cannot be assigned directly to an instance; it must be wrapped in an instance profile first.
`health_check_type = "ELB"`	The Auto Scaling Group asks the *load balancer*'s health check result to decide if an instance is healthy, rather than only the more basic EC2-level check (is the instance merely running). An instance that's up but failing application-level health checks gets replaced either way.
Target tracking scaling policy, 60% CPU	Rather than defining explicit scale-out/scale-in step thresholds by hand, target tracking lets AWS automatically calculate and adjust the necessary scaling actions to hold average CPU utilization near the target value — the modern, recommended SAA-C03 default over manually-stepped policies.

EXAM ANGLE — WHY THE ASG SPANS BOTH AZS' APP SUBNETS

`vpc_zone_identifier = aws_subnet.app[*].id` passes *both* AZs' app subnet IDs to the ASG, letting it distribute instances across both automatically. This is the direct mechanism behind "Auto Scaling Groups are Multi-AZ by design" — a frequently tested SAA-C03 fact that only holds true if the ASG is actually configured to span multiple subnets in different AZs, which it must be told to do explicitly.

EXPECTED RESULT (WERE THIS EVER APPLIED)

```
aws elbv2 describe-target-health --target-group-arn <arn>
```

Would show 2 healthy targets (the ASG's desired capacity) distributed across both AZs, each passing the `/health` check on port 8443.

8. Objective 5 — RDS Multi-AZ, Read Replica & Secrets Manager

`rds.tf`

AWS SAA-C03: Domain 1.0 (Design Resilient Architectures) | Security+ SY0-701: Domain 3.0 (Secrets/credential management)

Purpose: a highly-available relational database tier where master credentials never appear in plaintext anywhere in the configuration, and read-heavy workloads (reporting/analytics) don't compete with the primary for capacity.

Credentials, word by word

```
resource "random_password" "db_master" {
  length  = 24
  special = true
  override_special = "!#$%^&*()-_=+[]{}<>:?"
}

resource "aws_secretsmanager_secret_version" "db_credentials" {
  secret_string = jsonencode({
    username = var.db_master_username
    password = random_password.db_master.result
  })
}
```

Term	Meaning
`random_password`	Generates the master password at `apply` time — it is never typed by a human, never committed to source control, and never appears in the Terraform configuration files themselves; only the generated *result* exists, and only inside Terraform state and Secrets Manager.
`override_special`	RDS disallows a handful of special characters in its master password (notably `/`, `@`, `"`, and space) — this argument restricts the generated password's character set to only symbols RDS actually accepts, so `terraform apply` doesn't fail on an invalid password after the fact.
Secrets Manager, not a plain output/variable	The app tier's EC2 instances retrieve the password by calling the Secrets Manager API at runtime (using the IAM role from Objective 4), rather than the password ever being baked into an AMI, user data, or environment file directly.

Primary & read replica, word by word

```
resource "aws_db_instance" "primary" {
  storage_encrypted    = true
  kms_key_id           = aws_kms_key.main.arn
  multi_az             = true
  publicly_accessible = false
  backup_retention_period = 7
  deletion_protection    = true
}

resource "aws_db_instance" "read_replica" {
  replicate_source_db = aws_db_instance.primary.identifier
  availability_zone    = var.availability_zones[1]
}
^ rds gated behind LocalStack's paid tier - verified only via `terraform plan`, not `apply`.
See Section 3.1. random_password/Secrets Manager above (Objective 5's credential handling)
WERE verified with a real `apply`.
```

Term	Meaning
`multi_az = true`	Provisions a **synchronous** standby replica in the second AZ automatically. On a failure of the primary, RDS performs an automatic failover to the standby, updating the DNS-based endpoint so the application doesn't need any manual reconnection logic — this standby is not used for read traffic; it exists purely for availability.
Read replica (separate resource, `replicate_source_db`)	Uses **asynchronous** replication instead, and unlike the Multi-AZ standby, a read replica *can* serve live read-only queries — used here to offload reporting/analytics queries from the primary so they don't compete with production transaction traffic for I/O capacity.
`storage_encrypted` + `kms_key_id`	Encrypts the underlying storage at rest using the customer-managed KMS key from Objective 7 — the same key protecting the S3 audit logs, giving one consistent encryption boundary across the whole audit/data trail rather than a separate default AWS-managed key per service.
`publicly_accessible = false`	The RDS instance receives no public endpoint at all — reachable only from inside the VPC (specifically, only from the app tier's Security Group), consistent with the DB subnet's complete lack of an internet route from Objective 2.
`deletion_protection = true`	A Terraform-level and AWS-level safeguard requiring this flag be explicitly turned off before the instance can be destroyed at all — protects the production database tier against an accidental `terraform destroy` or console deletion.

EXAM ANGLE — MULTI-AZ VS. READ REPLICA, THE SINGLE MOST TESTED RDS DISTINCTION

> Multi-AZ is exclusively about **availability** (synchronous standby, automatic failover, same region); a read replica is exclusively about **read scalability** (asynchronous, can be cross-region, must be manually promoted to become writable). They solve different problems and are not substitutes for each other — this design uses both simultaneously precisely because it needs both properties.

EXPECTED RESULT (WERE THIS EVER APPLIED)

```
aws rds describe-db-instances --db-instance-identifier arehi-secops-db-primary
```

Would report `MultiAZ: true`, a `SecondaryAvailabilityZone` matching AZ-b, and `StorageEncrypted: true` referencing the KMS key from Objective 7.

9. Objective 6 — AWS WAF (SQLi/XSS + Rate Limiting)

`waf.tf`

AWS SAA-C03: Domain 3.0 (Design Secure Applications and Architectures)

Security+ SY0-701: Domain 4.5 (Web/application attacks — SQLi, XSS)

Purpose: a Layer 7 defense sitting directly in front of the ALB, inspecting the actual content of HTTP requests (headers, body, query strings) — a fundamentally different layer than the Security Groups/NACLs from Objective 3, which only ever look at Layer 3/4 (IP address, port), and have no visibility into what a request's body actually contains.

Web ACL, word by word

```
resource "aws_wafv2_web_acl" "main" {
  scope = "REGIONAL"
  default_action { allow {} }

  rule {
    name     = "AWS-AWSManagedRulesSQLiRuleSet"
    priority = 1
    override_action { none {} }
    statement {
      managed_rule_group_statement {
        name        = "AWSManagedRulesSQLiRuleSet"
        vendor_name = "AWS"
      }
    }
  }

  rule {
    name     = "rate-limit-per-ip"
    priority = 3
    action { block {} }
    statement {
      rate_based_statement {
        limit              = 2000
        aggregate_key_type = "IP"
      }
    }
  }
}

resource "aws_wafv2_web_acl_association" "alb" {
  resource_arn = aws_lb.main.arn
  web_acl_arn  = aws_wafv2_web_acl.main.arn
}
```

^ wafv2 gated behind LocalStack's paid tier - verified only via `terraform plan`, not `apply`. See Section 3.1.

Term	Meaning
`scope = "REGIONAL"`	Required for attaching a Web ACL to an ALB (or API Gateway) — the alternative, `CLOUDFRONT` scope, applies only to a CloudFront distribution and is a global rather than per-region resource. Using the wrong scope for the target resource type is a real, common misconfiguration.
AWS Managed Rule Groups (`AWSManagedRulesSQLiRuleSet`, `AWSManagedRulesCommonRuleSet`)	Pre-built, AWS-maintained pattern sets for known attack signatures — SQL injection patterns and general common web exploits (which includes cross-site scripting) respectively. Using managed rule groups means the signature set is kept current by AWS without this project needing to write or maintain its own regex-based detection rules.
`override_action { none {} }`	Tells the Web ACL to respect whatever action each managed rule inside the group already specifies (mostly `block`) rather than overriding every one of them to only `count` (log-only, no actual blocking) — `none` here means "don't override," which is somewhat counterintuitively the setting that makes the rule group actually enforce.
Rate-based rule	A separate, custom rule (not a managed group) blocking any single source IP once it exceeds 2000 requests in a rolling 5-minute window — a basic Layer 7 brute-force/DoS mitigation independent of the two signature-based managed rule groups.
`priority`	Rules are evaluated in ascending priority order; the first rule that both matches and takes a terminating action (like `block`) stops further evaluation for that request.
`aws_wafv2_web_acl_association`	The Web ACL and the ALB are two independent resources until explicitly linked by this association resource — omitting it would leave a fully-configured Web ACL that filters nothing at all, since it isn't actually attached to any traffic path.

EXAM ANGLE — WAF VS. SECURITY GROUP, KNOW WHICH LAYER CATCHES WHAT

A Security Group would never catch a SQL injection payload — from a Security Group's point of view, a malicious `' OR 1=1--` in a request body is indistinguishable from any other allowed TCP packet on port 443. WAF is the only control in this entire design operating at the layer where request *content* itself is inspected, which is precisely why it exists as an additional layer rather than a replacement for anything in Objective 3.

EXPECTED RESULT (WERE THIS EVER APPLIED)

CloudWatch metrics (enabled via `visibility_config` on every rule) would show a `BlockedRequests` count incrementing under the `sqli-rule-set` and `common-rule-set` metric names whenever a

malicious request pattern is actually matched.

10. Objective 7 — KMS & Encrypted S3 Audit Logging

`logging.tf`

AWS SAA-C03: Domain 3.0 (Design Secure Applications and Architectures)

Security+ SY0-701: Domain 3.0 (Data protection — encryption at rest, integrity)

Purpose: a single, KMS-encrypted, versioned S3 bucket that every other audit trail in this phase writes into — ALB access logs (Objective 4) land here, and RDS (Objective 5) is encrypted using the same KMS key, giving one consistent encryption boundary across the design rather than several independent default keys.

KMS key & bucket, word by word

```
resource "aws_kms_key" "main" {
  deletion_window_in_days = 30
  enable_key_rotation     = true
}

resource "aws_s3_bucket_public_access_block" "audit_logs" {
  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}

resource "aws_s3_bucket_versioning" "audit_logs" {
  versioning_configuration { status = "Enabled" }
}
```

Term	Meaning
`enable_key_rotation = true`	AWS automatically rotates the underlying cryptographic key material yearly while keeping the key's ARN/ID stable — old data encrypted under a previous rotation remains decryptable transparently, without any re-encryption step or application-visible change.
`deletion_window_in_days = 30`	A mandatory waiting period before a scheduled KMS key deletion actually takes effect — deleting a KMS key permanently destroys the ability to decrypt anything encrypted under it, so this window exists specifically as a safeguard against an irreversible mistake.
Public access block — all 4 flags `true`	Blocks every mechanism by which an S3 bucket could become publicly accessible (new ACLs, existing ACLs, new bucket policies, and cross-account access) simultaneously. This directly defends against the single most common real-world S3 misconfiguration — an accidentally public bucket — and is set at the bucket level so no single future policy change can silently override it.
Versioning	An accidental delete or overwrite of an audit log object doesn't actually lose the original — it creates a new version while the prior one remains retrievable, matching the general principle that audit records should be effectively immutable.
Lifecycle rule (Glacier at 90 days, expire at 2555 days)	Older logs automatically transition to cheaper cold storage rather than remaining on standard storage indefinitely, and are retained for roughly 7 years before final expiration — a realistic financial/audit-record retention period, cost-optimization applied to a security control (SAA-C03 Domain 4.0) without weakening it.

Bucket policy — granting the ALB permission to write logs, word by word

```
data "aws_elb_service_account" "main" {}

resource "aws_s3_bucket_policy" "audit_logs" {
  policy = jsonencode({
    Statement = [
      {
        Sid       = "AllowALBLogDelivery"
        Principal = { AWS = data.aws_elb_service_account.main.arn }
        Action    = "s3:PutObject"
      },
      {
        Sid    = "DenyUnencryptedTransport"
        Effect = "Deny"
        Condition = { Bool = { "aws:SecureTransport" = "false" } }
      }
    ]
  })
}
```

Term	Meaning
`data "aws_elb_service_account"`	A data source (not a resource — nothing is created) that looks up the AWS-owned account ID responsible for delivering ELB/ALB access logs in the current region. ALB access logging requires the bucket policy to explicitly trust this specific AWS-managed principal before it will accept log deliveries at all.
`DenyUnencryptedTransport` statement	An explicit `Deny` (not merely the absence of an allow) for any request made without TLS (`aws:SecureTransport = false`) — forces every interaction with this bucket, including from within AWS's own network, to happen only over an encrypted connection.

EXPECTED RESULT (WERE THIS EVER APPLIED)

```
aws s3api get-bucket-encryption --bucket <bucket>
aws s3api get-public-access-block --bucket <bucket>
```

Would confirm `aws:kms` as the default SSE algorithm referencing the KMS key above, and all 4 public-access-block flags set to `true` .

11. Objective 8 — Site-to-Site IPsec VPN

`vpn-gateway.tf` — full IPsec Phase 1/2 parameter reference in `03-aws-cloud-infrastructure/vpn-gateway/README.md`

CCNA 200-301: Domain 1.0 & 4.0 (WAN/hybrid connectivity) AWS SAA-C03: Domain 1.0 (Hybrid architecture connectivity)

Purpose: connects this VPC back to the ASEAN on-prem network built in Phases 1–3, over an encrypted tunnel across the public internet rather than a private leased line — the direct AWS-side completion of the WAN interface `SG-EDGE-GW` was configured with all the way back in Phase 1.

Resource blocks, word by word

```
resource "aws_customer_gateway" "sg_edge_gw" {
  bgp_asn    = var.on_prem_bgp_asn
  ip_address = var.on_prem_customer_gateway_ip
  type       = "ipsec.1"
}

resource "aws_vpn_gateway" "main" {
  vpc_id = aws_vpc.main.id
}

resource "aws_vpn_connection" "main" {
  customer_gateway_id = aws_customer_gateway.sg_edge_gw.id
  vpn_gateway_id      = aws_vpn_gateway.main.id
  static_routes_only  = true
}

resource "aws_vpn_connection_route" "on_prem" {
  destination_cidr_block = var.on_prem_cidr
}
^ create_vpn_connection_route is simply "not implemented" in LocalStack, independent of
license tier (unlike the elbv2/rds/wafv2 gaps above, which are paid-tier gated). Everything
above this line WAS verified with a real `apply`, including 2 route-propagation resources not
shown here (same "not implemented" gap).
```

Term	Meaning
Customer Gateway (CGW)	AWS's representation of the *on-prem* end of the tunnel — here, `SG-EDGE-GW` 's public-facing IP (`203.0.113.2` , a Packet Tracer/RFC 5737 placeholder — see Section 3) and its BGP ASN.
Virtual Private Gateway (VGW)	The AWS-managed, redundant VPN concentrator on the *AWS* side, attached directly to the VPC — the direct counterpart to the CGW.
`static_routes_only = true`	Chosen because `SG-EDGE-GW` 's Phase 1–3 configuration already uses static `ip route` statements on its WAN-facing side rather than a dynamic routing protocol — the AWS side is configured to match rather than introducing BGP dynamic routing on only one end of the tunnel.
Two tunnels per connection	AWS provisions exactly 2 tunnel endpoints per VPN connection by default, each terminating at a different AWS public IP, specifically so a maintenance event or failure on one AWS-side endpoint doesn't take the whole hybrid connection down — the same "no single point of failure" principle already applied to the per-AZ NAT Gateways.

WHY THIS TUNNEL CAN NEVER ACTUALLY COME UP — RESTATED FROM SECTION 3

`SG-EDGE-GW` 's configured public IP is an RFC 5737 documentation address, not a real routable one — IKE negotiation has no real internet path to travel across. This resource set is genuinely correct Terraform that would establish a real tunnel against a real on-prem endpoint; the gap is entirely the lab's own inherent boundary, not an error in either side's configuration.

EXPECTED RESULT (WERE THIS EVER APPLIED, AGAINST A REAL ENDPOINT)

```
aws ec2 describe-vpn-connections --vpn-connection-ids <id>
```

Would report `Tunnel1Status: UP` / `Tunnel2Status: UP` once IKE Phase 1 and IPsec Phase 2 negotiation both succeed on the real on-prem router's matching configuration — see the vpn-gateway README for the exact IKE/IPsec parameters (AES-256, SHA-256, DH Group 14) that endpoint would need to match.

12. Design Decisions & Honesty Notes

Decision	Reasoning
Never run `terraform apply`	No live AWS account exists behind this project, and every hourly-billed resource here (NAT Gateways, RDS Multi-AZ + replica, EC2, ALB) would accrue real ongoing cost regardless of actual traffic — stated explicitly rather than silently avoided
One NAT Gateway per AZ, not one shared	A single shared NAT Gateway would make every private subnet's internet egress depend on one AZ, undermining the entire point of spanning 2 AZs elsewhere in the design
DB subnets have zero internet route	Removes an entire class of exfiltration risk by not providing a path at all, rather than relying solely on Security Group/NACL rules to block it after the fact
Multi-AZ RDS *and* a read replica, not just one	They solve genuinely different problems — synchronous-standby availability vs. asynchronous read scalability — and this design needs both properties simultaneously
Both Security Groups and NACLs, not just one	Deliberately demonstrates the stateful-vs-stateless distinction directly, and provides real defense-in-depth against either layer being individually misconfigured
VPN Gateway configured despite never being able to establish a live tunnel	The AWS-side configuration is genuinely correct and complete; the on-prem peer's placeholder IP was never going to be internet-reachable regardless of how Phase 4 was built, a limitation inherited from the Packet Tracer lab itself, not from this Terraform

13. Appendix — Quick Reference Tables

What to check	Command (if ever applied)
Configuration validity	`terraform validate`, `terraform plan`
Target group health	`aws elbv2 describe-target-health --target-group-arn <arn>`
RDS Multi-AZ / encryption status	`aws rds describe-db-instances --db-instance-identifier <id>`
S3 encryption / public access block	`aws s3api get-bucket-encryption`, `get-public-access-block`
WAF blocked-request metrics	CloudWatch, namespace `AWS/WAFV2`
VPN tunnel status	`aws ec2 describe-vpn-connections --vpn-connection-ids <id>`

File	Key resources
`vpc.tf`	`aws_vpc.main`, `aws_subnet.public/app/db`, `aws_nat_gateway.main`
`security-groups.tf`	`aws_security_group.alb/app/db`, `aws_network_acl.public/app/db`
`alb.tf`	`aws_lb.main`, `aws_autoscaling_group.app`, `aws_iam_role.app_instance`
`rds.tf`	`aws_db_instance.primary/read_replica`, `aws_secretsmanager_secret.db_credentials`
`waf.tf`	`aws_wafv2_web_acl.main`, `aws_wafv2_web_acl_association.alb`
`logging.tf`	`aws_kms_key.main`, `aws_s3_bucket.audit_logs`
`vpn-gateway.tf`	`aws_customer_gateway.sg_edge_gw`, `aws_vpn_gateway.main`, `aws_vpn_connection.main`
